

Домашняя 1. Redis

Описание задачи	2
Архитектура системы	2
Функциональные требования	3
Инфраструктурные требования	4
Рекомендуемый контракт эндпоинтов	5
Требования к демонстрации	6
Формат сдачи	6
Скрипт самопроверки	7
Критерии оценки	7

Описание задачи

Вы разрабатываете бэкенд для платформы «GameHub» — сервиса, который управляет игровыми турнирами, рейтингами игроков, достижениями и уведомлениями. Сервис должен использовать Redis как хранилище и кеш¹.

Вам предстоит спроектировать и реализовать систему, используя ключевые возможности Redis: различные типы данных (строки, хеши, множества, сортированные множества, потоки), репликацию, Sentinel для высокой доступности, кеширование и конвейеризацию.

Важно: для операций, требующих атомарности (инкремент счётчика с установкой TTL), разрешается использовать Lua-скрипты. Для всех остальных операций используйте стандартные команды Redis.

Архитектура системы

Компоненты системы

1. Приложение (Application) — REST API сервис для управления игроками, турнирами и достижениями. Взаимодействует с Redis через Sentinel.
2. Redis Master — основной узел, принимающий операции записи. Настроен с персистентностью (RDB + AOF), политикой вытеснения и защитой от split-brain.
3. Redis Replicas (2 шт.) — реплики мастера, обслуживающие запросы на чтение.
4. Redis Sentinel (3 шт.) — процессы для автоматического обнаружения отказа и переключения мастера.
5. Redis Insight (опционально) — визуальный интерфейс для мониторинга.

Форматы данных

Тип данных	Структура Redis	Поля / Значение	TTL	Описание
Профиль игрока	Hash	name, level, region, created_at	∞	Основные данные игрока
Счётчик входов	String	числовое значение	24 часа	INCR при каждом

¹ В этой домашней работе Redis выполняет роль единственного хранилища — это сознательное упрощение, чтобы вы сосредоточились на механике Redis и не тратили время на настройку реляционной базы. В продакшене данные с TTL = ∞ (профили, лидерборды, достижения) жили бы в основной транзакционной базе, а в Redis хранились бы только кеш, сессии и счётчики

				входе (атомарно)
Кеш профиля	String	JSON-строка с данными профиля	60 сек	Быстрый доступ
Лидерборд	Sorted Set	member = player_id, score = очки	∞	Рейтинг игроков
Достижения	Set	строки с названиями достижений	∞	Уникальные достижения игрока
Очередь уведомлений	Stream	поля: player_id, type, message, timestamp	7 дней	Поток уведомлений

Функциональные требования

1. Управление профилями игроков (Hash)

1. Создание/обновление профиля: `HSET player:{id} name {name} level {level} region {region} created_at {timestamp}`
2. Получение профиля: `HGETALL player:{id}`
3. Обновление уровня: `HINCRBY player:{id} level {delta}`

2. Счётчик входов (String + INCR + Lua-скрипт)

1. При каждом входе игрока увеличивать счётчик `logins:{id}` на 1.
2. Счётчик должен автоматически сбрасываться через 24 часа.
3. Используйте Lua-скрипт для атомарного выполнения `INCR` и установки TTL при первом входе (или обновления TTL при каждом входе).
4. Проверка: `GET logins:{id}` должен показывать количество входов за день.

3. Лидерборд турнира (Sorted Set)

1. Турнир идентифицируется как `tournament:main`.
2. Добавление/обновление очков: `ZADD` или `ZINCRBY tournament:main {score} {player_id}`
3. Получение топ-10 игроков: `ZREVRANGE tournament:main 0 9 WITHSCORES`
4. Получение места игрока: `ZRANK tournament:main {player_id}`

4. Достижения игроков (Set)

1. У каждого игрока множество достижений: `achievements:{id}`
2. Добавление достижения: `SADD achievements:{id} {achievement_name}`
3. Проверка наличия: `SISMEMBER achievements:{id} {achievement_name}`
4. Пересечение достижений двух игроков: `SINTER achievements:{id1} achievements:{id2}`

5. Кеширование профилей (Cache-Aside + TTL)

1. При запросе профиля сначала читать из кеша: `GET cache:player:{id}`
2. Если кеш пуст — читать профиль из хеша (`HGETALL player:{id}`), сериализовать в JSON и записать в кеш с `TTL = 60` секунд.
3. При обновлении профиля – инвалидировать кеш (`DEL cache:player:{id}`).

6. Очередь уведомлений (Streams)

1. При изменении профиля (например, повышении уровня) отправлять уведомление в поток `notifications` через `XADD`.
2. Не забудьте инициализировать группу потребителей: `XGROUP CREATE notifications notifications-group $ MKSTREAM`
3. Реализовать одного потребителя, который читает уведомления через `XREADGROUP`, обрабатывает их (выводит в консоль) и подтверждает через `XACK`.
4. Сообщения должны храниться в потоке 7 дней (настроить `MAXLEN` или политику удаления).

7. Конвейеризация (Pipeline)

1. При инициализации системы (или для массовой загрузки начальных данных) использовать конвейер для отправки нескольких команд без ожидания ответа на каждую.
2. Например, создать 10–20 профилей одной пачкой через `pipeline`.

Инфраструктурные требования

1. Docker Compose

Запустить отказоустойчивую конфигурацию Redis через Docker Compose:

1. 1 мастер (порт 6379) с персистентностью (RDB + AOF)
2. 2 реплики (порты 6380, 6381) с настройкой `replicaof`
3. 3 Sentinel (порты 26379, 26380, 26381) с кворумом 2
4. Redis Insight (порт 5540) для визуального мониторинга (опционально)

Подсказки по настройке:

1. Для мастера: включить `appendonly yes`, настроить `save` для RDB, установить `maxmemory` и политику вытеснения `volatile-lru`, а также:
 - o `min-replicas-to-write 1`
 - o `min-replicas-max-lag 10`

Это защищает от `split-brain` (мастер с живыми репликами принимает записи, изолированный мастер — нет).

2. Для реплик: указать `replicaof` для подключения к мастеру, разрешить только чтение (`replica-read-only yes`).
3. Для Sentinel: настроить `sentinel monitor mymaster redis-master 6379 2`, задать `down-after-milliseconds`, `failover-timeout`, `resolve-hostnames yes`, `announce-hostnames yes`.
4. В Docker Compose использовать общую сеть для взаимодействия контейнеров.

2. Приложение

Разработать REST API сервис на любом языке программирования, примеры ниже:

Язык	Рекомендуемые библиотеки	Веб-фреймворк
Python	<code>redis</code> (с поддержкой Sentinel)	FastAPI / Flask
Java	<code>Lettuce</code> или <code>Jedis</code> (с Sentinel)	Spring Boot
Go	<code>go-redis</code> (с <code>FailoverClient</code>)	Gin / Echo

Подсказки по подключению:

1. Приложение должно подключаться к Redis через Sentinel, а не напрямую к мастеру.
2. Использовать встроенную поддержку Sentinel в библиотеке.
3. Настроить таймауты и ретраи при подключении.
4. Реализовать проверку соединения при старте.

3. Персистентность и политики вытеснения

1. Мастер: `appendonly yes`, `appendfsync everysec`, `auto-aof-rewrite-percentage 100`.
2. `maxmemory` установить на разумный лимит (например, 256 МБ).
3. `maxmemory-policy = volatile-lru`.

Рекомендуемый контракт эндпоинтов

Метод	Путь	Что делает	Операции в Redis
POST	<code>/api/players/{id}</code>	Создать/обновить профиль	<code>HSET player:{id} name {name} level {level} region {region} created_at {ts}</code>
GET	<code>/api/players/{id}</code>	Получить профиль (с кешем)	Кеш: <code>GET cache:player:{id}</code> , при промахе: <code>HGETALL player:{id} + SET cache:player:{id} {json} EX 60</code>
PATCH	<code>/api/players/{id}/level</code>	Обновить уровень	<code>HINCRBY player:{id} level {delta} + DEL cache:player:{id} + XADD notifications * ...</code>
POST	<code>/api/players/{id}/login</code>	Зафиксировать вход	Lua-скрипт: <code>INCR logins:{id} + EXPIRE logins:{id} 86400</code>

POST	/api/leaderboard/score	Добавить/обновить очки	ZINCRBY tournament:main {score} {player_id}
GET	/api/leaderboard/top?limit=10	Топ-10 игроков	ZREVRANGE tournament:main 0 9 WITHSCORES
GET	/api/leaderboard/rank/{playerId}	Место игрока	ZRANK tournament:main {player_id}
POST	/api/players/{id}/achievements	Добавить достижение	SADD achievements:{id} {achievement_name}
GET	/api/players/{id}/achievements/{name}	Проверить наличие достижения	SISMEMBER achievements:{id} {achievement_name}
GET	/api/players/{id1}/achievements/common/{id2}	Общие достижения двух игроков	SINTER achievements:{id1} achievements:{id2}
POST	/api/players/batch	Массовое создание профилей	Конвейер: пачка HSET через pipeline

Требования к демонстрации

Подготовить видео-демонстрацию (5–10 минут), показывающую:

1. Запуск системы – `docker compose up -d`, проверка статуса контейнеров, `INFO replication` показывает 2 реплики.
2. Работа приложения – создание профиля, обновление уровня, добавление очков в лидерборд, получение топ-10, добавление достижений, пересечение достижений.
3. Кеширование – первый запрос профиля (cache miss), повторный (cache hit), проверка TTL.
4. Очередь уведомлений – при обновлении профиля уведомление появляется в потоке; потребитель читает и подтверждает.
5. Отказоустойчивость (Sentinel) – остановка мастера (`docker stop redis-master`), ожидание ~15 секунд, проверка, что Sentinel переключил мастера на реплику, и приложение продолжает работать.
6. Защита от split-brain – покажите, что при `docker pause` на всех репликах мастер начинает отклонять записи (ошибка `min-replicas-to-write`). Затем восстановите реплики (`docker unpause`).
7. Конвейеризация – показ массовой загрузки (например, 20 профилей) через pipeline.

Формат сдачи

1. Ссылка на репозиторий с:
 - Исходным кодом приложения, `docker-compose.yml` и конфигурационными файлами (`redis.conf`, `sentinel.conf`)
 - `README.md` с инструкцией по запуску и описанием архитектуры
2. Видео-демонстрация (5–10 минут) с показом всех сценариев из требований выше.

Скрипт самопроверки

Перед сдачей запустите скрипт `dzl_check.py`. Он проверит основные требования и выдаст отчёт.

Критерии оценки

№	Критерий	Баллы	Описание и способ проверки
1	Инфраструктура Docker Compose	2	Все контейнеры (мастер, 2 реплики, 3 Sentinel) запускаются без ошибок. Проверка: <code>docker compose ps</code> – все Up.
2	Репликация и Sentinel	2	Репликация работает: <code>connected_slaves:2</code> . При остановке мастера Sentinel переключает на реплику за ~15 сек. Проверка: <code>SENTINEL get-master-addr-by-name mymaster</code> показывает новый адрес.
3	Функциональность: профили, лидерборд, достижения	2	Работают все операции: создание/обновление профиля, лидерборд (топ-10, место), достижения (добавление, проверка, пересечение). Проверка через API.
4	Кеширование	1	Кеш работает: первый запрос – miss, второй – hit, TTL установлен. Проверка: два запроса подряд, <code>TTL cache:player:{id} > 0</code> .
5	Очередь уведомлений (Streams)	1	При обновлении профиля уведомление добавляется в <code>notifications</code> . Потребитель читает и подтверждает (XACK). Проверка: <code>XLEN notifications > 0</code> после операции.
6	Конвейеризация	1	Массовая загрузка данных (не менее 10 записей) через <code>pipeline</code> выполнена. Проверка: код использует <code>pipeline()</code> , замер времени.
7	Защита от split-brain	0.5	На мастере настроены <code>min-replicas-to-write 1</code> . При <code>docker pause</code> на всех репликах мастер отклоняет записи. Проверка: демонстрация.
8	Документация и качество кода	0.5	README содержит инструкцию по запуску, описание API, структуру данных. Код читаемый, есть комментарии.

Итого: 10 баллов